ASSIGNMENT 30

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

https://github.com/omkulawade03/Frontend-Session-Assignments/tree/main/task30

Q1. (Environment Setup with uv)

Install uv on your system.

Create a new virtual environment using uv venv.

Activate the virtual environment.

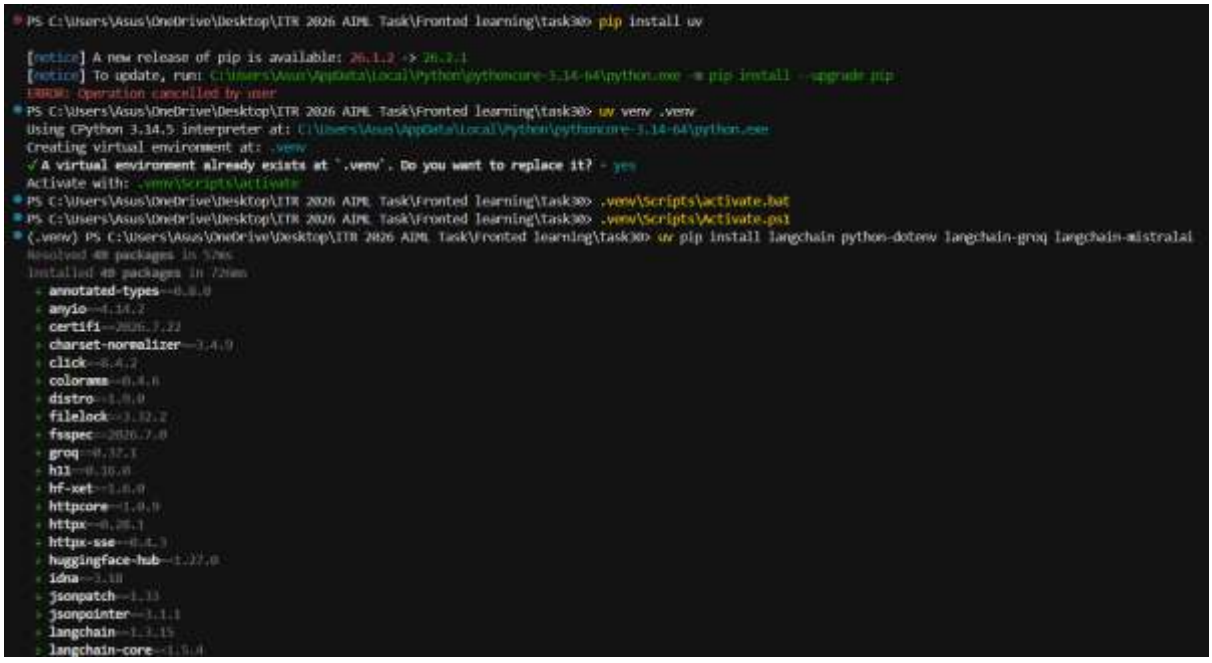
Install the following packages using uv: langchain, python-dotenv, langchain-

groq, and langchain-mistralai.

Show the terminal commands and confirm successful installation.

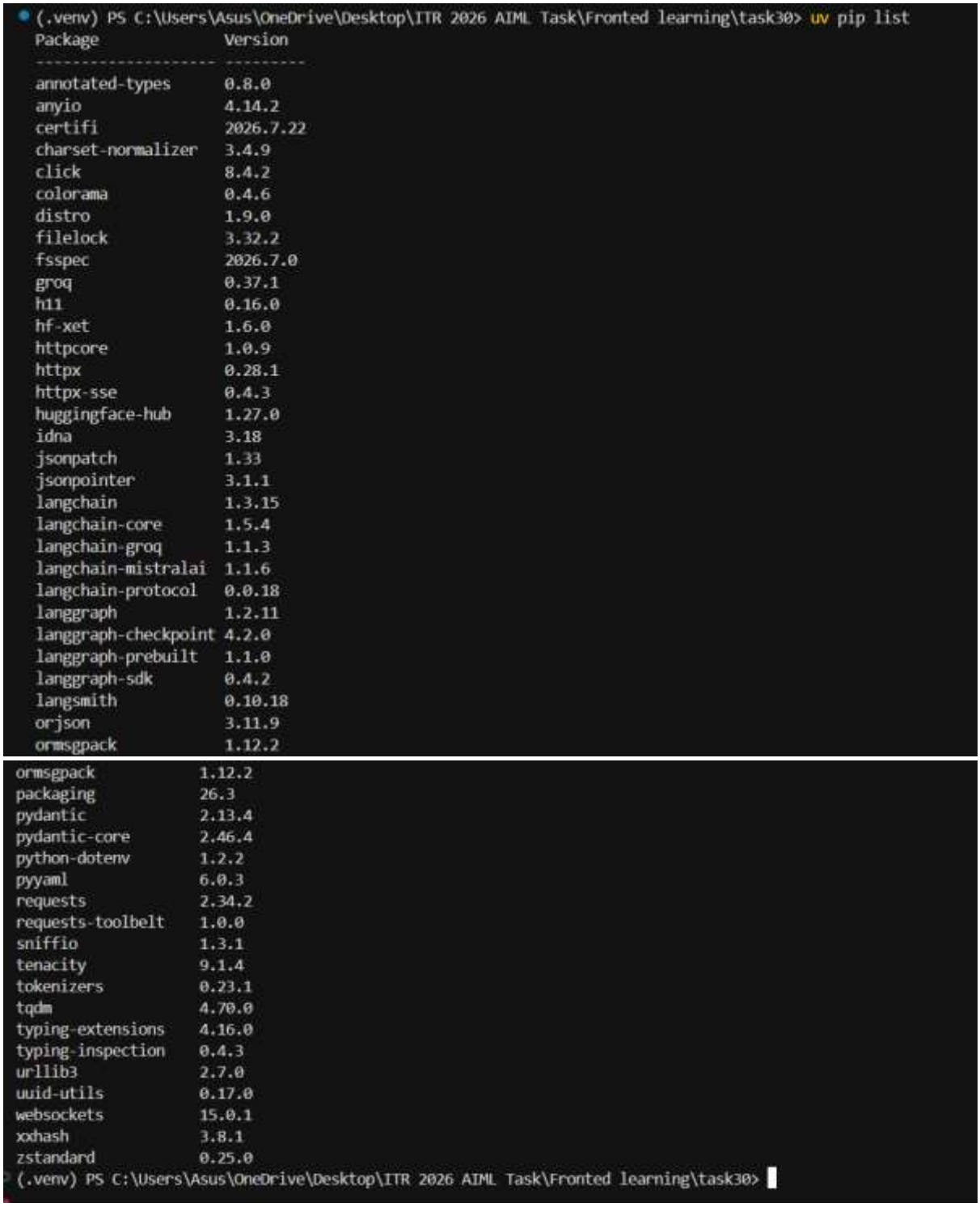
Program Code:

#Questions 01




+ langchain-groq==1.1.1
+ langchain-mistralai==1.1.6
+ langchain-protocol==0.0.10
+ langgraph==1.2.11
+ langgraph-checkpoint==4.2.0
+ langgraph-prebuilt==1.1.0
+ langgraph-sdk==0.4.2
+ langsmith==0.10.10
+ orjson==3.11.9
+ ormsgpack==1.12.2
+ packaging==26.3
+ pydantic==2.13.4
+ pydantic-core==2.46.4
+ python-dotenv==1.2.2
+ pyyaml==6.0.3
+ requests==2.34.2
+ requests-toolbelt==1.0.0
+ sniffio==1.3.1
+ tenacity==9.1.4
+ tokenizers==0.23.1
+ tqdm==4.70.0
+ typing-extensions==4.16.0
+ typing-inspection==0.4.3
+ urllib3==2.7.0
+ uuid-utils==0.17.0
+ websockets==15.0.1
+ xxhash==3.8.1
+ zstandard==0.25.0

OUTPUT SCREENSHOT:



```
(.venv) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task30> uv pip list
Package            Version
------------------ ---------
annotated-types    0.8.0
anyio              4.14.2
certifi            2026.7.22
charset-normalizer 3.4.9
click              8.4.2
colorama           0.4.6
distro             1.9.0
filelock           3.32.2
fsspec             2026.7.0
groq               0.37.1
h11                0.16.0
hf-xet             1.6.0
httpcore           1.0.9
httpx              0.28.1
httpx-sse          0.4.3
huggingface-hub    1.27.0
idna               3.18
jsonpatch          1.33
jsonpointer        3.1.1
langchain          1.3.15
langchain-core     1.5.4
langchain-groq     1.1.3
langchain-mistralai 1.1.6
langchain-protocol 0.0.18
langgraph          1.2.11
langgraph-checkpoint 4.2.0
langgraph-prebuilt 1.1.0
langgraph-sdk      0.4.2
langsmith          0.10.18
orjson             3.11.9
ormsgpack          1.12.2
ormsgpack          1.12.2
packaging          26.3
pydantic           2.13.4
pydantic-core      2.46.4
python-dotenv      1.2.2
pyyaml             6.0.3
requests           2.34.2
requests-toolbelt  1.0.0
sniffio            1.3.1
tenacity           9.1.4
tokenizers         0.23.1
tqdm               4.70.0
typing-extensions  4.16.0
typing-inspection  0.4.3
urllib3            2.7.0
uuid-utils         0.17.0
websockets         15.0.1
xxhash             3.8.1
zstandard          0.25.0
(.venv) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task30>
```

Q2. (API Key Management)

Create a .env file in your project folder and store the following keys:

GROQ_API_KEY

MISTRAL_API_KEY (or GOOGLE_API_KEY if using Gemini)

Write Python code to load the environment variables using python-dotenv and

print a confirmation message that the keys are loaded successfully (do not print the actual keys).

Program Code:

#Questions 02

load_env.py > ...

```python
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

# Verify keys are loaded without printing their sensitive values
groq_key = os.getenv("GROQ_API_KEY")
mistral_key = os.getenv("MISTRAL_API_KEY")

if groq_key and mistral_key:
    print("Success: API keys loaded successfully!")
else:
    print("Warning: One or more API keys are missing.")

```

OUTPUT SCREENSHOT:

Q3. (Basic LLM Call with Groq)

Using LangChain and the Groq model llama-3.1-8b-instant:

Initialize the chat model.

Send the prompt: "Introduce yourself in 3 sentences."

Print the response content.

Program Code:

#Questions 03

q3_groq.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq

# Load environment variables from .env
load_dotenv()

# Initialize the chat model using the specified model name
llm = ChatGroq(model="llama-3.1-8b-instant")

# Send the prompt
response = llm.invoke("Introduce yourself in 3 sentences.")

# Print the response content clearly
print("Groq Response:\n", response.content)

```

OUTPUT SCREENSHOT:

Q4. (Controlling Output with Parameters)

Modify the Groq model from Q3 by setting:

temperature=0.2

max_tokens=50

Send the same prompt and compare the response with the previous one. Write your observation.

Program Code:

#Questions 04

q4_params.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq

# Load environment variables from .env
load_dotenv()

# Initialize the Groq model with modified parameters
llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0.2,
    max_tokens=50
)

# Send the same prompt as Q3
response = llm.invoke("Introduce yourself in 3 sentences.")

# Print the constrained response content
print("Constrained Groq Response:\n", response.content)

```

OUTPUT SCREENSHOT:

Q5. (Using Mistral Model)

Initialize the Mistral model mistral-small-2603 using LangChain.

Send the prompt: "Explain what is Artificial Intelligence in simple words."

Print the complete response.

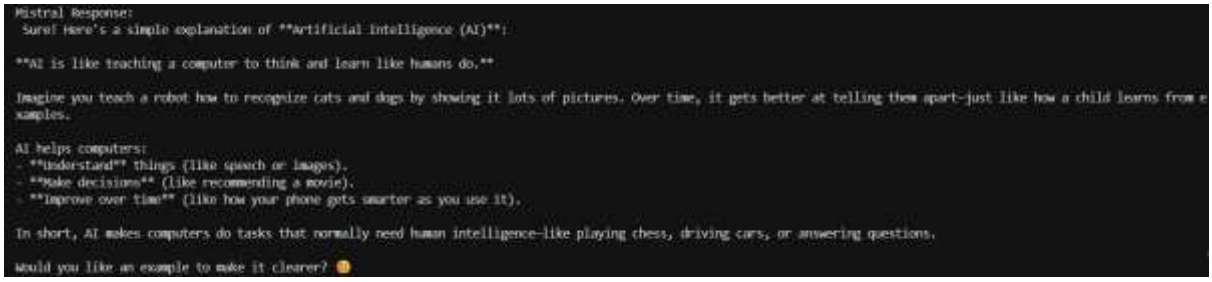
Program Code:

#Questions 05

q5_mistral.py > ...

```python
from dotenv import load_dotenv
from langchain_mistralai import ChatMistralAI

# Load environment variables from .env
load_dotenv()

# Initialize the Mistral model using LangChain
llm = ChatMistralAI(model="mistral-small-2603")

# Send the prompt
response = llm.invoke("Explain what is Artificial Intelligence in simple words.")

# Print the complete response
print("Mistral Response:\n", response.content)

```

OUTPUT SCREENSHOT:



Q6. (Comparing Two Models)

Write a program that asks the same question to both Groq (llama-3.1-8b-instant)

and Mistral (mistral-small-2603) models:

Question: "What are the advantages of using LangChain?"

Print both responses clearly labeled as “Groq Response” and “Mistral Response”.

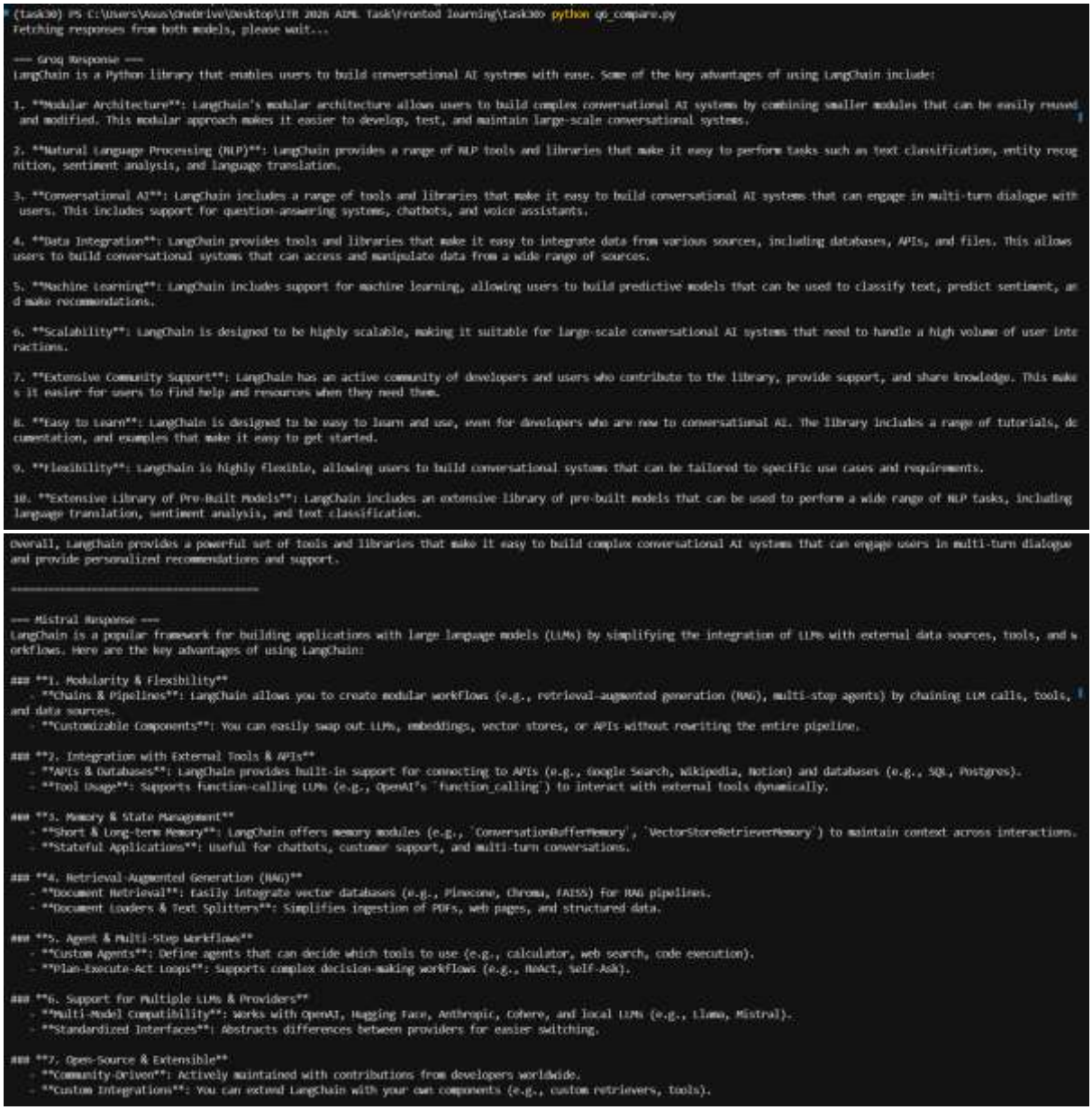
Program Code:

```
#Questions 06
```

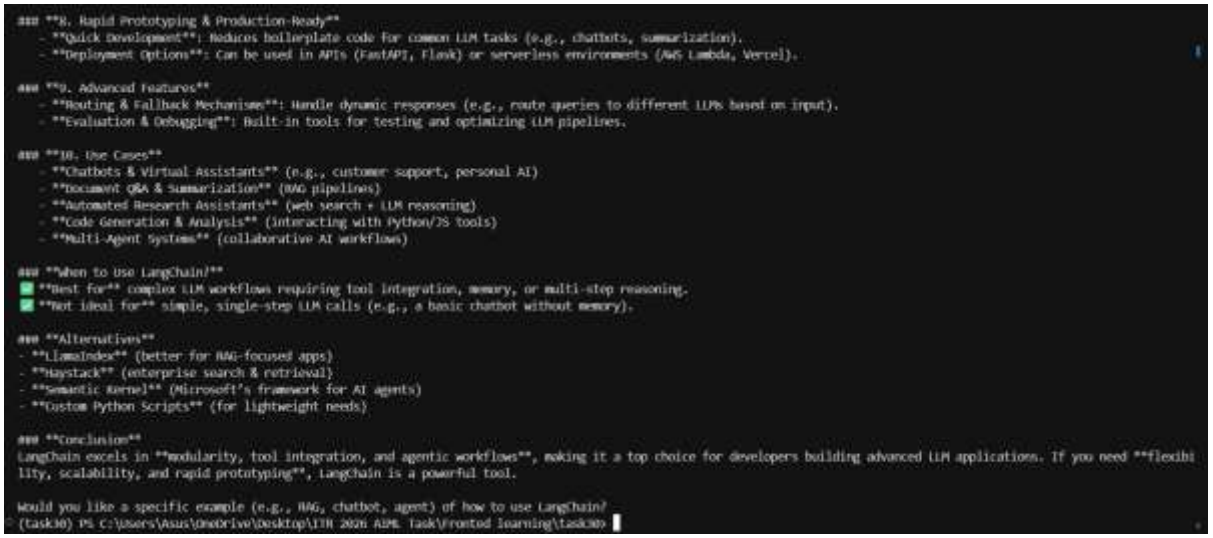
q6_compare.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_mistralai import ChatMistralAI

# Load environment variables from .env
load_dotenv()

# The same question for both models
question = "What are the advantages of using LangChain?"

# Initialize both models
groq_llm = ChatGroq(model="llama-3.1-8b-instant")
mistral_llm = ChatMistralAI(model="mistral-small-2603")

print("Fetching responses from both models, please wait...\n")

# Get responses
groq_response = groq_llm.invoke(question)
mistral_response = mistral_llm.invoke(question)

# Print clearly labeled responses
print("=== Groq Response ===")
print(groq_response.content)

print("\n" + "="*40 + "\n")

print("=== Mistral Response ===")
print(mistral_response.content)

```

OUTPUT SCREENSHOT:



```
(task30) PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Frontend learning\task30> python qb_compare.py
Fetching responses from both models, please wait...

=== Groq Response ===
LangChain is a Python library that enables users to build conversational AI systems with ease. Some of the key advantages of using LangChain include:

1. **Modular Architecture**: LangChain's modular architecture allows users to build complex conversational AI systems by combining smaller modules that can be easily reused
 and modified. This modular approach makes it easier to develop, test, and maintain large-scale conversational systems.

2. **Natural Language Processing (NLP)**: LangChain provides a range of NLP tools and libraries that make it easy to perform tasks such as text classification, entity recog
nition, sentiment analysis, and language translation.

3. **Conversational AI**: LangChain includes a range of tools and libraries that make it easy to build conversational AI systems that can engage in multi-turn dialogue with
 users. This includes support for question-answering systems, chatbots, and voice assistants.

4. **Data Integration**: LangChain provides tools and libraries that make it easy to integrate data from various sources, including databases, APIs, and files. This allows
users to build conversational systems that can access and manipulate data from a wide range of sources.

5. **Machine Learning**: LangChain includes support for machine learning, allowing users to build predictive models that can be used to classify text, predict sentiment, an
d make recommendations.

6. **Scalability**: LangChain is designed to be highly scalable, making it suitable for large-scale conversational AI systems that need to handle a high volume of user inte
ractions.

7. **Extensive Community Support**: LangChain has an active community of developers and users who contribute to the library, provide support, and share knowledge. This make
s it easier for users to find help and resources when they need them.

8. **Easy to Learn**: LangChain is designed to be easy to learn and use, even for developers who are new to conversational AI. The library includes a range of tutorials, do
cumentation, and examples that make it easy to get started.

9. **Flexibility**: LangChain is highly flexible, allowing users to build conversational systems that can be tailored to specific use cases and requirements.

10. **Extensive Library of Pre-Built Models**: LangChain includes an extensive library of pre-built models that can be used to perform a wide range of NLP tasks, including
 language translation, sentiment analysis, and text classification.

Overall, LangChain provides a powerful set of tools and libraries that make it easy to build complex conversational AI systems that can engage users in multi-turn dialogue
and provide personalized recommendations and support.

==========

=== Mistral Response ===
LangChain is a popular framework for building applications with large language models (LLMs) by simplifying the integration of LLMs with external data sources, tools, and w
orkflows. Here are the key advantages of using LangChain:

### **1. Modularity & Flexibility**
   - **Chains & Pipelines**: LangChain allows you to create modular workflows (e.g., retrieval-augmented generation (RAG), multi-step agents) by chaining LLM calls, tools, a
nd data sources.
   - **Customizable Components**: You can easily swap out LLMs, embeddings, vector stores, or APIs without rewriting the entire pipeline.

### **2. Integration with External Tools & APIs**
   - **APIs & Databases**: LangChain provides built-in support for connecting to APIs (e.g., Google Search, Wikipedia, Notion) and databases (e.g., SQL, Postgres).
   - **Tool Usage**: Supports function-calling LLMs (e.g., OpenAI's `function_calling`) to interact with external tools dynamically.

### **3. Memory & State Management**
   - **Short & Long-term Memory**: LangChain offers memory modules (e.g., `ConversationBufferMemory`, `VectorStoreRetrieverMemory`) to maintain context across interactions.
   - **Stateful Applications**: Useful for chatbots, customer support, and multi-turn conversations.

### **4. Retrieval-Augmented Generation (RAG)**
   - **Document Retrieval**: Easily integrate vector databases (e.g., Pinecone, Chroma, FAISS) for RAG pipelines.
   - **Document Loaders & Text Splitters**: Simplifies ingestion of PDFs, web pages, and structured data.

### **5. Agent & Multi-Step Workflows**
   - **Custom Agents**: Define agents that can decide which tools to use (e.g., calculator, web search, code execution).
   - **Plan-Execute-Act Loops**: Supports complex decision-making workflows (e.g., ReAct, Self-Ask).

### **6. Support for Multiple LLMs & Providers**
   - **Multi-Model Compatibility**: Works with OpenAI, Hugging Face, Anthropic, Cohere, and local LLMs (e.g., Llama, Mistral).
   - **Standardized Interfaces**: Abstracts differences between providers for easier switching.

### **7. Open-Source & Extensible**
   - **Community-Driven**: Actively maintained with contributions from developers worldwide.
   - **Custom Integrations**: You can extend LangChain with your own components (e.g., custom retrievers, tools).
```


8. Rapid Prototyping & Production-Ready
- **Quick Development**: Reduces boilerplate code for common LLM tasks (e.g., chatbots, summarization).
- **Deployment Options**: Can be used in APIs (FastAPI, Flask) or serverless environments (AWS Lambda, Vercel).
9. Advanced Features
- **Routing & Fallback Mechanisms**: Handle dynamic responses (e.g., route queries to different LLMs based on input).
- **Evaluation & Debugging**: Built-in tools for testing and optimizing LLM pipelines.
10. Use Cases
- **Chatbots & Virtual Assistants** (e.g., customer support, personal AI)
- **Document QA & Summarization** (RAG pipelines)
- **Automated Research Assistants** (web search + LLM reasoning)
- **Code Generation & Analysis** (interacting with Python/JS tools)
- **Multi-Agent Systems** (collaborative AI workflows)
When to Use LangChain?
✅ **Best for** complex LLM workflows requiring tool integration, memory, or multi-step reasoning.
✅ **Not ideal for** simple, single-step LLM calls (e.g., a basic chatbot without memory).
Alternatives
- **LlamaIndex** (better for RAG-focused apps)
- **Haystack** (enterprise search & retrieval)
- **Semantic Kernel** (Microsoft's framework for AI agents)
- **Custom Python Scripts** (for lightweight needs)
Conclusion
LangChain excels in **modularity, tool integration, and agentic workflows**, making it a top choice for developers building advanced LLM applications. If you need **flexibi
lity, scalability, and rapid prototyping**, LangChain is a powerful tool.
Would you like a specific example (e.g., RAG, chatbot, agent) of how to use LangChain?
(task30) PS C:\Users\Asus\OneDrive\Desktop\ITH 2026 AIML Task\Frontod learning\task30>

Q7. (Temperature Experiment)

Using the Groq model, send the same creative prompt three times with different

temperature values:

temperature=0.1

temperature=0.7

temperature=1.2

Prompt: "Write a short creative story about a robot learning to cook."

Observe and write how the creativity of the output changes.

Program Code:

#Questions 07

q7_temp.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq

# Load environment variables
load_dotenv()

prompt = "Write a short creative story about a robot learning to cook."
temperatures = [0.1, 0.7, 1.2]

for temp in temperatures:
    llm = ChatGroq(model="llama-3.1-8b-instant", temperature=temp)
    response = llm.invoke(prompt)
    print(f"--- Temperature: {temp} ---")
    print(response.content)
    print("\n" + "="*40 + "\n")

```

OUTPUT SCREENSHOT:

Q8. (Simple Chat Loop)

Create a simple command-line chatbot using the Groq model:

Continuously take user input.

Send the input to the model and print the response.

Exit the loop when the user types "exit".

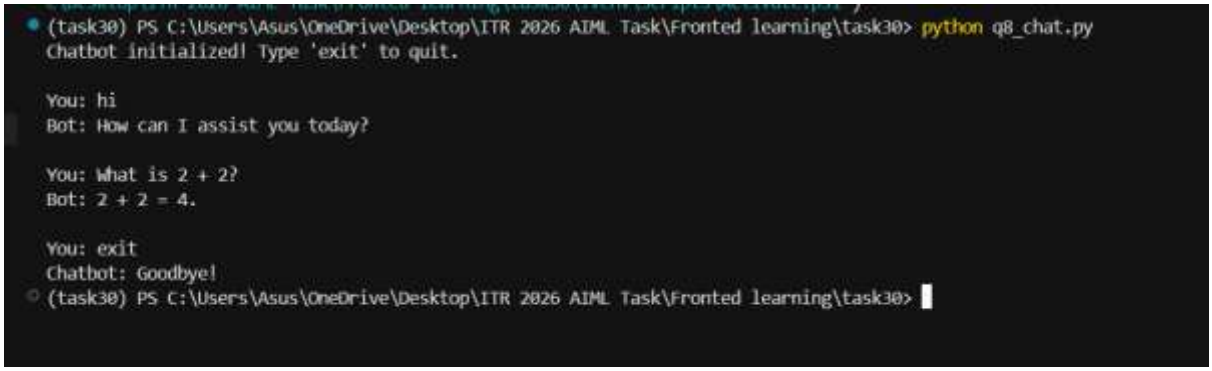
Program Code:

#Questions 08

q8_chat.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq

# Load environment variables
load_dotenv()

# Initialize the chat model
llm = ChatGroq(model="llama-3.1-8b-instant")

print("Chatbot initialized! Type 'exit' to quit.\n")

while True:
    user_input = input("You: ")

    # Check for exit condition
    if user_input.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    # Get and print response
    response = llm.invoke(user_input)
    print(f"Bot: {response.content}\n")

```

OUTPUT SCREENSHOT:



Q9. (Structured Prompting)

Create a function that takes a topic as input and uses the LLM to generate:

1. A short definition

2. Three key points

3. One real-life example

Use a well-structured prompt and the Groq model. Test it with the topic "Machine

Learning".

Program Code:

```
#Questions 09
```

q9_memory.py > ...

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# Load environment variables
load_dotenv()

# Initialize the model
llm = ChatGroq(model="llama-3.1-8b-instant")

# Store session histories in a dictionary
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# Wrap the model with message history handling
chat_with_history = RunnableWithMessageHistory(
    llm,
    get_session_history
)

# Configuration for the session
config = {"configurable": {"session_id": "user_session_1"}}

print("Memory Chatbot initialized! Type 'exit' to quit.\n")

while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    # Invoke the model with history config
    response = chat_with_history.invoke(user_input, config=config)
    print(f"Bot: {response.content}\n")
```

OUTPUT SCREENSHOT:

Q10. (Mini Project – Multi-Model Assistant)

Build a small Python application that:

1. Loads API keys from .env

2. Allows the user to choose between Groq or Mistral model

3. Takes a user question

4. Displays the model's response

5. Continues until the user types "quit"

Add proper error handling if the API key is missing or the model fails.

Program Code:

#Questions 10

```python
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# Load environment variables
load_dotenv()

# Initialize the model
llm = ChatGroq(model="llama-3.1-8b-instant")

# Create a prompt template with a system persona
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a sarcastic and witty AI assistant. Always add a witty remark before answering any question."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])

# Combine prompt and model into a chain
chain = prompt | llm

# Store session histories
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
```

```python

# Wrap the chain with message history
chat_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)

# Configuration for the session
config = {"configurable": {"session_id": "user_session_1"}}

print("Persona Chatbot initialized! Type 'exit' to quit.\n")

while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    # Invoke the chain with history config
    response = chat_with_history.invoke({"input": user_input}, config=config)
    print(f"Bot: {response.content}\n")

```

OUTPUT SCREENSHOT: